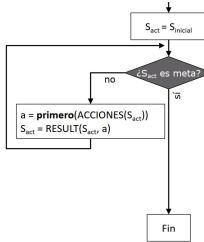


Acción	Precondiciones	Resultado
Llenar A	(x,y) con $x \geq 8$	$(8,y)$
Llenar B	(x,y) con $y \geq 6$	$(x,6)$
Vaciar A	(x,y) con $x \geq 0$	$(0,y)$
Vaciar B	(x,y) con $y \geq 0$	$(x,0)$
Vaciar A en B	(x,y) con $x \geq 0, y \geq 6, x + y \leq 6$	$(0,x+y)$
Vaciar B en A	(x,y) con $x \geq 8, y \geq 0, x + y \leq 8$	$(x+y,0)$
Llenar A con B	(x,y) con $x \geq 8, y \geq 0, x + y > 8$	$(8,x+y-8)$
Llenar B con A	(x,y) con $x \geq 0, y \geq 6, x + y > 6$	$(x+y-6,6)$

Estrategia básica 1 (Ingenua)



1. $S_{actual} = S_{inicial}$
2. Prueba de meta
Si S_{actual} es meta $\rightarrow$ Fin
Si no $\rightarrow$ Seguir
3. Aplicar 1ª acción aplicable a S_{actual}
4. Volver a 2

Simplemente aplica la primera acción disponible y avanza.

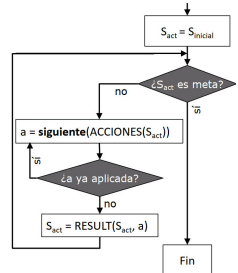
- Fallo: Caen en bucles infinitos (ej. llenar A $\rightarrow$ vaciar A $\rightarrow$ llenar A...). No tiene memoria.

Genera: (0,0) Llenar_A (8,0) Llenar_B (8,6) Vaciar_A (0,6) Llenar_A (8,6) ..

- Fallo: Aún puede bloquearse o no encontrar solución si se queda sin acciones nuevas en un estado.

Estrategia básica 2 (Memoria de acciones)

1. $S_{actual} = S_{inicial}$
2. Prueba de meta
Si S_{actual} es meta $\rightarrow$ Fin
Si no $\rightarrow$ Seguir
3. $a = \text{siguiente acción aplicable a } S_{actual}$
4. ¿a ya aplicada?
5. Volver a 2

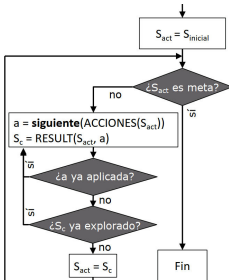


Genera: (0,0) Llenar_A (8,0) Llenar_B (8,6) Vaciar_A (0,6) Vaciar_B (0,0)

Verifica si el estado resultante ya ha sido explorado antes.

- Fallo: Puede quedarse sin opciones si entra en un callejón sin salida y no sabe retroceder.

Estrategia básica 3 (Memoria de estados visitados)



1. $S_{actual} = S_{inicial}$
2. Prueba de meta
Si S_{actual} es meta $\rightarrow$ Fin
Si no $\rightarrow$ Seguir
3. $a = \text{siguiente acción aplicable a } S_{actual}$
4. ¿a ya aplicada?
5. ¿RESULT(S_{actual} , a) ya explorado?
6. Volver a 2

Genera: (0,0) Llenar_A (8,0) Llenar_B (8,6) Vaciar_A (0,6) Vaciar_B_en_A (6,0)

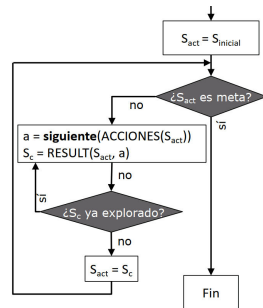
Si un camino no funciona, vuelve al estado anterior y prueba otra opción.

- Éxito: Encuentra solución, pero no garantiza que sea la óptima (la más corta o barata), ni que encuentre solución para cualquier problema

Genera: (0,0) Llenar_A (8,0) Llenar_B (8,6) Vaciar_A (0,6) Vaciar_B_en_A (6,0)
Llenar_B (6,6) Llenar_A_con_B (8,4) Vaciar_A (0,4) Vaciar_A_en_B (4,0)

Estrategia básica 4 (Sistemática con backtracking)

1. $S_{actual} = S_{inicial}$
2. Prueba de meta
Si S_{actual} es meta $\rightarrow$ Fin
Si no $\rightarrow$ Seguir
3. $a = \text{siguiente acción aplicable a } S_{actual}$
4. ¿RESULT(S_{actual} , a) ya explorado?
5. Volver a 2



Estrategias de búsqueda

- Progresiva (Forward/Data-driven): Del estado inicial a la meta. (Ej. ¿A dónde puedo ir desde aquí?).
- Regresiva (Backward/Goal-driven): De la meta al inicio. (Ej. ¿Desde dónde podría haber llegado a la meta?).

Características:

1. Dirección del proceso

- Progresiva (Forward/Data-driven): Del estado inicial a la meta. (Ej. ¿A dónde puedo ir desde aquí?).
- Regresiva (Backward/Goal-driven): De la meta al inicio. (Ej. ¿Desde dónde podría haber llegado a la meta?).

Decisión:

1. Número de estados iniciales vs. número de metas: De menor a mayor número de estados
2. Factor de ramificación, o número promedio de estados que podemos alcanzar directamente desde un estado dado: En el sentido del menor factor de ramificación
3. Inclusión de estructuras explicativas como requisito inicial en el diseño de nuestro sistema inteligente: De la forma que mejor se ajuste al modo de razonar de expertos y usuarios

2. Topología: ¿Árbol o grafo?

Árbol

función BUSQUEDA-ARBOL (problema) produce solución, o fallo
inicializar la frontera usando el estado inicial del problema
bucle hacer
si la frontera está vacía **entonces devolver fallo**
sino escoger un nodo hoja y eliminarlo de la frontera
si el nodo hoja contiene un estado meta, **entonces devolver la solución**
sino expandir el nodo elegido y añadir los nodos resultantes a la frontera
fin

Grafo

función BUSQUEDA-GRAFO (problema) produce solución, o fallo
inicializar la frontera usando el estado inicial del problema
inicializar el conjunto de explorados como vacío
bucle hacer
si la frontera está vacía **entonces devolver fallo**
sino escoger un nodo hoja y eliminarlo de la frontera
si el nodo hoja contiene un estado meta, **entonces devolver la solución**
sino añadir el nodo al conjunto de explorados
expandir el nodo elegido y añadir los nodos resultantes a la frontera
solo si no están en la frontera o conjunto de explorados
fin



No guarda memoria de estados visitados (cerrados). Si el espacio de estados tiene ciclos (bucles), el árbol de búsqueda puede ser infinito aunque el espacio sea finito.

Mantiene una lista de explorados (cerrados). Evita caminos redundantes y bucles.

La exploración sobre grafos

- reduce los esfuerzos de exploración del espacio de estados
- obliga a comprobar si cada estado generado ha sido explorado ya anteriormente
- demanda menos memoria, pero supone un coste computacional mayor

La exploración sobre árboles

- es más eficiente computacionalmente, pero demanda más memoria al almacenar nodos repetidos en la frontera

	Árbol	Grafo
Memoria	↑	↓
Eficiencia	↑	↓

3. Selección de Acciones (Emparejamiento / Matching)

Determinar que acciones son aplicables ("legales") en el estado actual

Acción	Precondiciones	Resultado
Llenar A	(x,y) con $x \neq 8$	$(8,y)$
Llenar B	(x,y) con $y \neq 6$	$(x,6)$
Vaciar A	(x,y) con $x \neq 0$	$(0,y)$
Vaciar B	(x,y) con $y \neq 0$	$(x,0)$
Vaciar A en B	(x,y) con $x \neq 0, y \neq 6, x+y \leq 6$	$(0,x+y)$
Vaciar B en A	(x,y) con $x \neq 8, y \neq 0, x+y \leq 6$	$(x+y,0)$
Llenar A con B	(x,y) con $x \neq 8, y \neq 0, x+y \leq 6$	$(8,x+y-6)$
Llenar B con A	(x,y) con $x \neq 0, y \neq 6, x+y \leq 6$	$(x+y,6)$

Progresivo

Acción	Precondiciones	Resultado
Llenar A	(x,y) con $x \neq 8$	$(8,y)$
Llenar B	(x,y) con $y \neq 6$	$(x,6)$
Vaciar A	(x,y) con $x \neq 0$	$(0,y)$
Vaciar B	(x,y) con $y \neq 0$	$(x,0)$
Vaciar A en B	(x,y) con $x \neq 0, y \neq 6, x+y \leq 6$	$(0,x+y)$
Vaciar B en A	(x,y) con $x \neq 8, y \neq 0, x+y \leq 6$	$(x+y,0)$
Llenar A con B	(x,y) con $x \neq 8, y \neq 0, x+y \leq 6$	$(8,x+y-6)$
Llenar B con A	(x,y) con $x \neq 0, y \neq 6, x+y \leq 6$	$(x+y,6)$

Regresivo

(0,0)

(4,0)

Resolución de conflictos, si varias acciones son posibles, ¿cuál elegimos?

Reglas de carácter general

1. Si podemos evitarlo no aplicaremos acciones ya aplicadas
2. Trataremos de aplicar primero acciones que lleven a estados nuevos
3. Aplicaremos primero operadores con precondiciones más restrictivas
4. De no poder discriminar con las pautas anteriores, ejecutaremos una (o más, depende de la estrategia), al azar

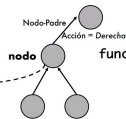
4. Uso de heurísticas

Uso de conocimiento específico ("pistas") para estimar qué tan cerca estamos de la meta. Fundamental para la eficiencia

Implementación: Estructuras de Datos Reconstrucción de la solución

7	2	4
5	3	6
8	3	1

estado



La exploración del espacio de estados producirá un árbol de búsqueda, que debemos representar. Llamaremos nodo a la representación de un estado en el árbol de búsqueda.

Para poder reconstruir la solución, cada nodo debe anotar su nodo padre y la acción que llevó hasta él.

función CREA_NODO(e, p, a)
 $n \leftarrow$ nuevo NODO
 $n.estado \leftarrow e$
 $n.padre \leftarrow p$
 $n.acción \leftarrow a$
devuelve n

función RECONSTRUYE_SOL(n)
 $solución \leftarrow \emptyset$
 $a \leftarrow n$
mientras $a \neq \emptyset$
 INSERT(a, solución)
 $a \leftarrow a.padre$
devuelve solución

Función sucesores

función SUCESORES(n)
 $sucesores \leftarrow \emptyset$
para cada ac, res aplicable a n.estado
 $n_h \leftarrow$ CREA_NODO(res, n, ac)
 INSERT(n_h , sucesores)
devuelve sucesores

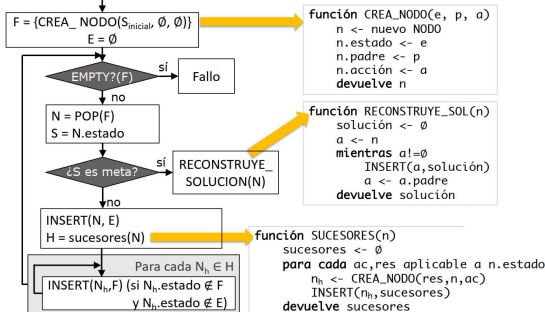
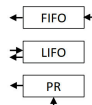
La función sucesores llevará a cabo la expansión de un nodo: aplicación de operadores relevantes y generación de sucesores

Listas

Frontera (Lista de Abiertos): Colección de nodos generados pero no expandidos (hojas del árbol). El tipo de estructura de datos de la Frontera define la estrategia:

- Cola FIFO (First-In, First-Out): Búsqueda en Anchura.
- Pila LIFO (Last-In, First-Out): Búsqueda en Profundidad.
- Cola de Prioridad: Búsqueda de Coste Uniforme o A* (Ordena por coste/importancia).

Lista de Explorados (Cerrados): Nodos ya expandidos (solo en búsqueda en Grafo). Idealmente una Tabla Hash para acceso rápido



Evaluación de estrategias

¿Cómo sabemos si un algoritmo es "bueno"? Usamos 4 métricas:

- Completitud: ¿Existencia solución. ¿La encuentra siempre?
- Optimización: ¿Encuentra la mejor solución (menor coste)?
- Complejidad Temporal: ¿Cuánto tarda? (max n° de nodos generados).
- Complejidad Espacial: ¿Cuánta RAM gasta? (max n° de nodos almacenados).

Variables:

- b: Factor de ramificación (cuántos hijos tiene cada nodo de media).
- d: Profundidad de la solución más superficial.
- m: Profundidad máxima del espacio de estados.

Coste total = coste búsqueda + coste camino

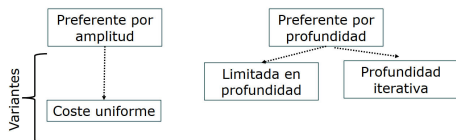
Tipos

No informada o ciega: no disponen de información adicional acerca de los estados más allá de la que proporciona la definición del problema

Informada o heurística: disponen de conocimiento específico del problema para alcanzar el objetivo de manera más eficiente

Estrategias de Búsqueda no Informada (Ciegas)

No saben "cuánto falta" para la meta. Solo saben generar sucesores y comprobar si son la meta.



A Búsqueda preferente en amplitud (breadth-first)

Frontera: **FIFO**

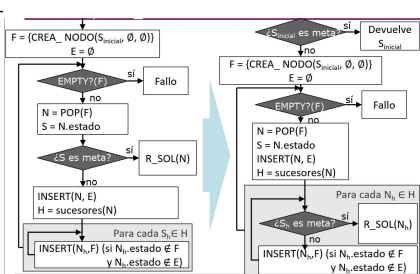
Utiliza una estructura de datos de cola FIFO (First-In, First-Out) para la lista de 'abiertos' (frontera). Esto significa que siempre expande el nodo más antiguo, lo que provoca que explore sistemáticamente todo un nivel de profundidad al completo antes de poder descender al siguiente nivel del árbol.

Mejora

El algoritmo clásico de Búsqueda en Grafo evalúa si un nodo es meta al momento de sacarlo de la frontera para expandirlo. La gran mejora para BFS es aplicar la prueba de meta en cuanto se generan los sucesores. Al hacerlo de esta manera precoz, no tenemos que añadir la meta a la frontera ni expandir inútilmente los miles de nodos hermanos que acompañan a ese nivel. Esta ligera modificación de código ahorra un nivel entero de generación, lo cual en crecimiento exponencial es colosal.

Evaluación

- Completitud: Sí. Siempre encontrará una solución si existe (siempre que el factor de ramificación sea finito).
- Optimalidad: Sí, siempre encuentra la ruta con el menor número de pasos. Pero atención: solo es estrictamente óptima si todas las acciones cuestan lo mismo (coste unitario).
- Complejidad Temporal: $O(b^d)$. Es lento para búsquedas profundas.
- Complejidad Espacial: $O(b^d)$. Este es su mayor defecto práctico: guarda todos los nodos generados en la memoria colapsando la RAM antes de que el procesador siquiera sufra.



B Búsqueda de coste uniforme (Dijkstra)

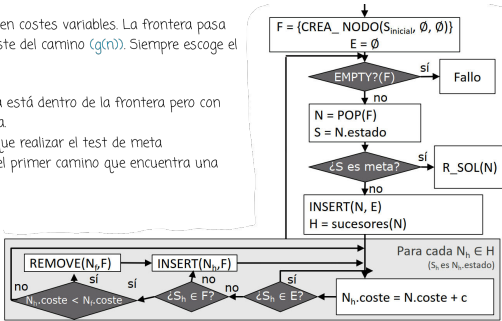
Es la adaptación de BFS cuando los arcos de nuestro grafo tienen costes variables. La frontera pasa de ser una cola FIFO a una cola de prioridad ordenada por el coste del camino ($g(n)$). Siempre escoge el camino acumulado más barato disponible.

Condiciones para garantizar optimalidad

Si al explorar encontramos un camino hacia un nodo que ya está dentro de la frontera pero con un coste menor, debemos actualizar ese coste en la frontera. Aquí NO se puede usar la variante rápida de Anchura Hay que realizar el test de meta obligatoriamente al extraer el nodo, no al generarlo, ya que el primer camino que encuentra una meta podría no ser el más barato.

Evaluación

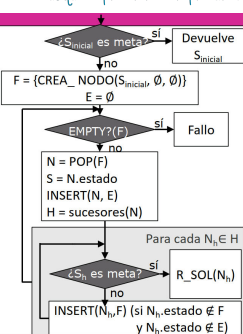
- Completitud: Sí si todos los costes son positivos (mayores que una constante E).
- Optimalidad: Sí, matemáticamente garantizado (se expande nodos en anillos concéntricos de coste ascendente).
- Complejidad Temporal: n^* nodos con coste de camino $\leq$ coste solución óptima $O(b^{n^*/E})$
- C^* coste solución óptima y E coste mínimo de cada acción
- Si coste de los pasos es el mismo $b^{n^*/E} = b^{n^*}$



No tiene el truco de testear al generar sino al expandir

C Búsqueda preferente en profundidad (depth-first)

Frontera: **LIFO**



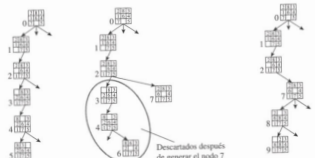
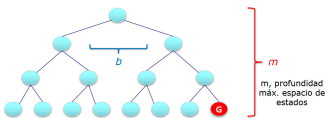
Su frontera es una pila LIFO (Last-In, First-Out). Expande el nodo recién entrado, lo que provoca que siga una rama y profundice constantemente hasta tocar un callejón sin salida (una hoja), momento en el cual realiza backtracking (retrocede al cruce anterior).

Evaluación

- Completitud: No, en árboles infinitos, ya que puede perderse viajando hasta el infinito en una rama errónea sin solución. (Es completa únicamente si el espacio está cerrado y es un grafo finito).
- Optimalidad: No. Puede encontrar en primer lugar una solución a 400 pasos de profundidad y devolverla felizmente, ignorando que en la rama contigua había una solución a 3 pasos.
- Complejidad Espacial: $O(b \cdot m)$. Este es su verdadero superpoder. Dado que solo necesita recordar el camino actual y los hermanos de esos nodos, gasta poquísima RAM, haciéndolo ideal frente a problemas masivos.

Mejora (Búsqueda con backtracking)

Podemos bajar el uso de memoria aún más de $O(b \cdot m)$ a simplemente $O(m)$ acciones. La idea es que, en vez de crear todos los sucesores posibles y añadirlos a la memoria, se genera un único sucesor a la vez modificando directamente el estado actual. Al terminar de explorar esa opción, deshacemos el cambio (volver atrás) y generamos la siguiente, por lo que el programa nunca carga hermanos enteros en RAM.

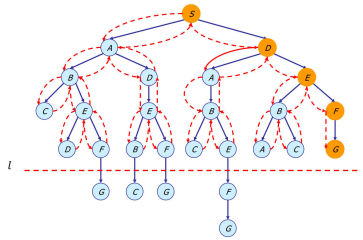


D. Búsqueda de profundidad limitada

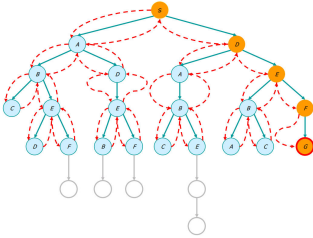
Para solucionar el defecto de DFS de perderse en ramas infinitas, cortamos su avance imponiendo artificialmente un límite l a la profundidad máxima del árbol.

Evaluación

- Si establecemos $l < d$ (el límite es demasiado corto y la meta estaba más abajo), fracasaremos y será un proceso incompleto.
- Si $l > d$, encontraremos metas pero perderemos la optimalidad (seguiremos pudiendo coger la ruta más profunda de todas).
- Complejidad temporal $O(b^l)$
- Complejidad espacial $O(b \cdot l)$



E. Búsqueda de profundidad iterativa



Ejecuta sucesivas búsquedas de profundidad limitada partiendo con un límite de $l=0$. Si falla, reinicia el sistema por completo y lo lanza con $l=1$, luego con $l=2$ etc.

El "mejor" de la búsqueda no informada:

Reúne todo lo bueno. Tiene la completitud y optimalidad (en pasos) del algoritmo BFS, junto con la espectacular y bajísima complejidad espacial del DFS.

¿No es un desperdicio gigantesco de tiempo recalcular la parte de arriba tantas veces? Intuitivamente parece malo, pero numéricamente no lo es. Debido a la naturaleza del ramaje, la abrumadora mayoría de los nodos siempre se generan en el último nivel. Por tanto, repetir la punta del iceberg no añade casi penalización real en tiempo.

	Amplitud	Coste Uniforme	Profundidad	Profundidad limitada	Profundización iterativa
Completa	Sí ^a	Sí ^{a,b}	No*	Sí ($l \geq d$)	Sí ^a
Óptima	Sí ^c	Sí	No	No	Sí ^c
Complejidad Espacial	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Complejidad Temporal	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$

b: factor de ramificación

d: profundidad de la solución menos profunda

m: máxima profundidad del árbol de búsqueda

l: límite de profundidad

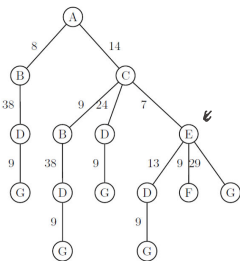
^a completa si b es finito

^b completa si coste de paso $\geq \epsilon$, $\epsilon > 0$

^c óptima si los costes de transición son iguales

* completa si EE finito y búsqueda grafo

Ejemplos



A. Anchura

Paso	Frontiera	Explorados
1	$A^{0,0}$	$A^{0,0}$
2	$B^{8,8}, C^{14,14}$	$A^{0,0}$
3	$D^{38,38}, E^{9,9}$	$A^{0,0}, B^{8,8}, C^{14,14}$
4	$F^{13,13}, G^{20,20}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}$
5	$G^{20,20}, F^{13,13}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}, F^{13,13}, G^{20,20}$

B. Coste uniforme

Paso	Frontiera	Explorados
1	$A^{0,0}$	$A^{0,0}$
2	$B^{8,8}, C^{14,14}$	$A^{0,0}$
3	$D^{38,38}, E^{9,9}$	$A^{0,0}, B^{8,8}, C^{14,14}$
4	$F^{13,13}, G^{20,20}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}$
5	$F^{13,13}, G^{20,20}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}, F^{13,13}, G^{20,20}$
6	$D^{38,38}, E^{9,9}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}, F^{13,13}, G^{20,20}$
7	$G^{20,20}, F^{13,13}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}, F^{13,13}, G^{20,20}$
8	$G^{20,20}, F^{13,13}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}, F^{13,13}, G^{20,20}$

C. Profundidad

Paso	Frontiera	Explorados
1	$A^{0,0}$	$A^{0,0}$
2	$B^{8,8}, C^{14,14}$	$A^{0,0}$
3	$D^{38,38}, E^{9,9}$	$A^{0,0}, B^{8,8}, C^{14,14}$
4	$G^{20,20}, F^{13,13}$	$A^{0,0}, B^{8,8}, C^{14,14}, D^{38,38}, E^{9,9}$

Estrategias de Búsqueda Informada (búsqueda heurística)

A diferencia de las búsquedas ciegas (que exploran por ensayo y error sistemático), las estrategias informadas sí poseen conocimiento específico acerca del problema. Saben evaluar si un estado es "más prometedor" que otro para llegar a la meta.

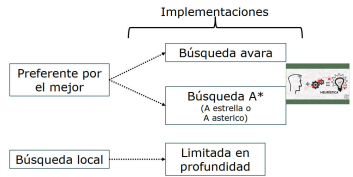
La función heurística

El núcleo de la búsqueda informada es la función heurística. Se define como una función matemática de carácter numérico que, dado un estado S , devuelve una estimación del coste del camino menos costoso para alcanzar la meta desde ese estado S .

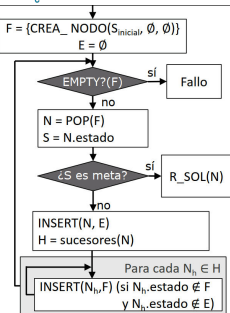
- Restricción inquebrantable: Si un nodo n es un estado meta, obligatoriamente su coste estimado debe ser cero: $h(n) = 0$.
- Propósito: Sirve para optimizar la búsqueda, guiando la exploración por el camino más provechoso y sugiriendo la mejor alternativa a priori cuando el algoritmo se topa con un cruce.

Formas de implementar la búsqueda por el mejor nodo:

- Si $f(n)$ sólo considera el coste mínimo estimado para llegar a una solución a partir de un nodo n , $h(n)$ o función heurística
 - $f(n) = h(n) \rightarrow$ **Búsqueda avara***
- Si $f(n)$ considera el coste mínimo total del camino a un nodo solución que pase por el nodo n
 - $f(n) = g(n) + h(n) \rightarrow$ **Búsqueda A* (A asterisco o estrella)**
 - $g(n) =$ coste consumido para llegar a n



Búsqueda Avara



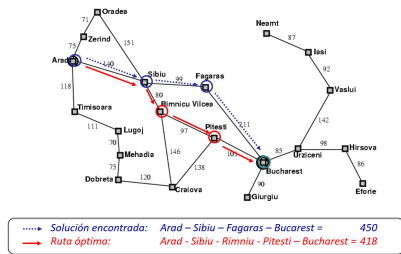
Si para evaluar los nodos solo nos fijamos de nuestra intuición del futuro, usamos la Búsqueda Avara.

- Fórmula: $f(n) = h(n)$
- Comportamiento: Expande siempre el nodo que parece estar más cerca de la meta
- Problema principal: Al ser "avaro" e ir ciegamente hacia lo que parece la meta, puede meterse en bucles o tomar caminos que, aunque inicialmente parecían directos, terminan siendo larguissimos laberintos (por ejemplo, tomar un atajo aparente que termina en un acantilado y obliga a dar un rodeo inmenso).

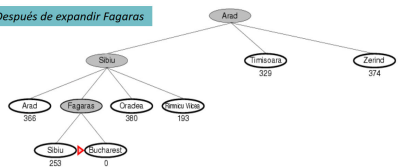
Frontera $\leftarrow$ PR(h)

Evaluación

- Óptima: NO. Es fácil que encuentre una solución subóptima al no considerar el coste ya recorrido.
- Completa: NO. En espacios infinitos o si no se controlan bien los repetidos, puede quedarse atascada en un bucle infinito. (Será completa solo en espacios finitos controlando los repetidos).
- Complejidad Espacial y Temporal: $O(b^m)$ (donde b es el factor de ramificación y m la profundidad máxima). Al igual que otros algoritmos en grafos, mantiene todos los nodos generados en memoria.



Después de expandir Fagaras



Solución encontrada Arad - Sibiu - Fagaras - Bucharest = 450

¡No puede ser mejor que la función de evaluación!

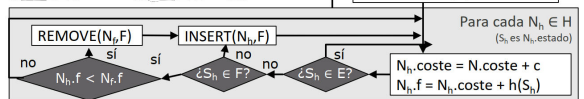
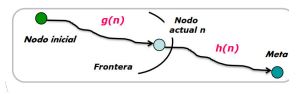
Función de evaluación Arad $\rightarrow$ Bucharest = 366

Búsqueda A*

Para solucionar la ineficiencia de la Búsqueda de Coste Uniforme (que es óptima pero ciega) y la seguridad al pasado de la Búsqueda Avara, nace A*.

Fórmula: $f(n) = g(n) + h(n)$

- $g(n)$: Coste real ya consumido para llegar desde el inicio hasta n
- $h(n)$: Coste estimado para ir desde n hasta la meta.
- $f(n)$: Coste estimado total del camino que pasa por el nodo n .



Equivalencias de A*

1. Búsqueda de Coste Uniforme:

$h(n) = 0$.

2. Búsqueda Avara (Greedy):

$g(n) = 0$

3. Búsqueda en Anchura:

g se incrementa en 1 (coste de paso = 1) y $h(n) = 0$.

Gestión de empates: Los nodos con igual valor f se ordenan de menos a más reciente (FIFO: First In, First Out).

Lógica: Como $h=0$ y cada paso suma 1, todos los nodos del nivel 1 tendrán $f=1$, los del nivel 2 tendrán $f=2$, etc. Para que explore estrictamente en anchura (todo el nivel 1 antes de tocar el nivel 2), cuando tenga varios nodos con el mismo f , debe elegir el más antiguo (el que llegó primero a la cola).

4. Búsqueda en Profundidad:

$g(n) = 0$ y $h(n) = 0$.

Gestión de empates: Los nodos se ordenan de más a menos reciente (LIFO: Last In, First Out).

Lógica: Si $g=0$ y $h=0$, entonces $f(n) = 0$ para absolutamente todos los nodos. El algoritmo está "ciego" matemáticamente. ¿Cómo decide entonces? Por el orden de llegada. Al coger siempre el más reciente (el último que se acaba de generar), se va hundiendo por una misma rama hasta el fondo, que es exactamente como funciona una pila en la búsqueda en profundidad.

Búsqueda A* óptima: Admisibilidad y consistencia

Para que A* nos devuelva garantizadamente la solución óptima, no sirve cualquier heurística. Debe cumplir requisitos matemáticos estrictos dependiendo de cómo se explore el espacio.

Heurísticas admisibles (para búsquedas en árboles)

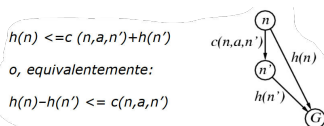
Una heurística es admisible si es puramente optimista. Es decir, nunca debe sobreestimar el coste real para llegar a la meta.

- Si el coste real es 100, la heurística puede decir 20, 50, o 100, pero jamás 101. Si dice 101, corremos el riesgo de descartar el camino óptimo creyendo falsamente que es muy caro.

Heurísticas consistentes (para búsquedas en grafos)

Dado que un grafo permite llegar a un mismo estado por múltiples caminos, la heurística debe ser consistente

- Fórmula matemática: $h(n) \leq c(n, a, n') + h(n')$.
- El coste estimado desde n a la meta no puede ser mayor que el coste real de dar un paso hacia su sucesor n' más el coste estimado desde n' a la meta.
- Esto garantiza la Monotonía: los valores de $f(n)$ a lo largo de cualquier camino nunca decrecerán. El primer nodo meta que seleccionemos para expandir será 100% el óptimo.



Evaluación de A*

- Óptima: Si. Ningún otro algoritmo es capaz de garantizar la expansión de menos nodos para encontrar el óptimo (es óptimamente eficiente).
- Completa: Sí (si el coste de paso es positivo y el factor b finito).
- Complejidad Temporal: exponencial
- Complejidad Espacial (el mayor problema): Guarda en memoria todos los nodos generados. A nivel computacional, un ordenador actual se quedará sin memoria (RAM) mucho antes de quedarse sin tiempo de procesamiento. No es apto para problemas increíblemente masivos.

Diseño de heurísticas admisibles

Sabemos que una función heurística $h(n)$ debe estimar el coste desde un estado actual S hasta la meta. Pero, ¿cómo nos inventamos una fórmula matemática que calcule eso sin resolver el problema entero por accidente?

Si calcular el coste exacto implicara resolver el problema P , la heurística no tendría sentido (tardaríamos lo mismo en calcular la pista que en encontrar la salida). La gran IDEA de la Inteligencia Artificial para resolver esto es la Relajación de Problemas.

¿Qué es un problema relajado P' ?

Consiste en tomar el problema original P al cual está sujeto a unas reglas o restricciones estrictas, y eliminarle (relajar) algunas de esas restricciones para crear un problema ficticio P' .

Al tener menos reglas, resolver P' es tan matemáticamente sencillo que podemos calcular su coste directo sin necesidad de hacer una búsqueda en árbol. Ese coste exacto de solucionar P' se convierte en nuestra heurística admisible y consistente para el problema original P .

Regla de Oro: Cuantas menos restricciones eliminemos, más difícil será calcular P' , pero más precisa y realista será nuestra heurística.

Ejemplo: el 8-Puzzle

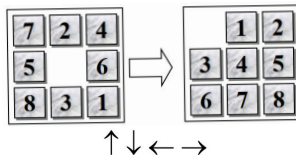
Si intentáramos resolver el 8-puzzle por búsqueda exhaustiva (a ciegas), nos enfrentaríamos a un árbol de unos 3×10^{10} estados generados. Necesitamos urgentemente una heurística para guiar al algoritmo A*.

Análisis de las Restricciones Originales:

Para mover una pieza de la casilla A a la casilla B, el mundo real nos impone dos leyes:

- A y B deben ser adyacentes (no puedo mover en diagonal ni saltar).
- La casilla B debe estar vacía (el hueco).

Vamos a crear universos paralelos (problemas relajados) eliminando reglas:

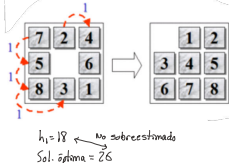


Universo A (Eliminamos la regla 2 - Permisitos solapamiento): "Puedo mover una pieza a una casilla adyacente, aunque haya otra pieza allí (las piezas se atraviesan como fantasmas)".

- Cálculo sin búsqueda: Cuento cuántos pasos horizontales y verticales separan a cada pieza de su meta, como si estuviera sola en el tablero, y los sumo.

- Nace la Heurística: h_1 = Distancia Manhattan (o suma de distancias de Hamming de cada pieza). (Ej. Puede darnos $h=18$).

- Admisibilidad: Es admisible porque en el mundo real, las piezas se estorban y tendrán que dar rodeos, por lo que el coste real SIEMPRE será mayor o igual a esta distancia ideal en línea recta ortogonal.



Universo B (Eliminamos la regla 1 - Permisitos teletransporte al hueco): "Puedo coger cualquier pieza de cualquier punta del tablero y meterla en el hueco libre mágicamente".

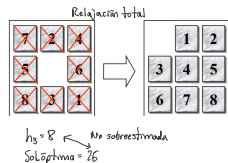
- Nace la Heurística: h_2 = Distancia de Gaschnig.

$h_2 = 6$ No subestimado
Sol. óptima = 18



Universo C (Eliminamos ambas reglas): "Puedo coger cualquier pieza y teletransportarla a cualquier casilla, esté ocupada o no".

- Cálculo sin búsqueda: Simplemente cuento cuántas piezas no están en su sitio.
- Nace la Heurística: h_3 = Número de piezas mal colocadas. (Ej. Si hay 8 mal colocadas, $h=8$).
- Admisibilidad: Es admisible porque, en el mundo real, cada pieza mal colocada tendrá que moverse, como mínimo, 1 vez.



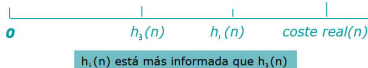
Dominancia entre heurísticas

Misma solución óptima, 26 pasos.

- h_3 (Piezas mal colocadas) estima: 8
- h_1 (Distancia Manhattan) estima: 18

Ambas son admisibles (ninguna supera a 26). Pero h_1 es mayor.

Matemáticamente, decimos que si para cualquier nodo n , $h_1(n) \geq h_3(n)$, entonces h_1 domina a h_3 . Una heurística dominante está 'más informada' (se pega más al coste real). Si usamos la heurística dominante en A*, el algoritmo garantiza expandir menos nodos inútiles, yendo mucho más directo a la solución.



Búsqueda local

Motivación: Cuando el viaje no importa, solo el destino

Hasta ahora, en algoritmos como A* o Anchura, el objetivo era descubrir la ruta (ej. "Ir de Arad a Bucarest"). El algoritmo guardaba celosamente el árbol de caminos en memoria.

Pero existen cientos de problemas en Ingeniería donde el camino nos da igual.

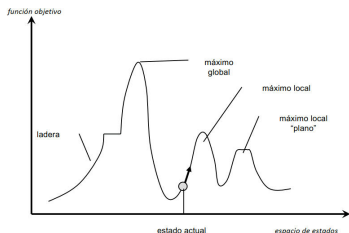
- Ejemplos: Configurar las 8 reinas en un tablero, diseñar la distribución de un microchip, crear los horarios de clases de la universidad.
- Nadie te pregunta "qué pasos diste para hacer el horario", solo te exigen "dame un horario válido y óptimo".

Para estos casos nace la Búsqueda Local.

- Gestión de Memoria: Extrema eficiencia. Solo guardan en memoria un único estado (el estado actual) y evalúan a sus vecinos inmediatos. La complejidad espacial pasa a ser una cantidad constante (O(1)).

- Enfoque: Son algoritmos diseñados para la Optimización. Buscan el "mejor" estado posible evaluado mediante una función objetivo.

Algoritmo de escalada (Hill-climbing)



Cada punto representa un estado y valor de la función objetivo.

Se trata de encontrar el punto más alto, máximo global.

A efectos prácticos, consideramos que el algoritmo intenta maximizar una función objetivo, en el caso de función heurística, en realidad intenta minimizar la función heurística.

función HILL-CLIMBING (problema) devuelve un estado que es un máximo local
entradas: un problema
variables locales: actual, un nodo vecino, un nodo

bucle hacer

actual <- HACER_NODO(ESTADO_INICIAL[problema])
si VALOR[vecino] <= VALOR[actual]
entonces devolver ESTADO[actual]
actual <- vecino

Steepest ascent (máxima pendiente)

En caso de varios elegir uno aleatoriamente

Inconvenientes

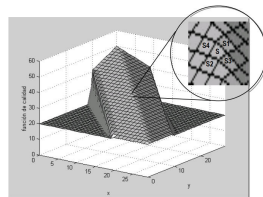
Como el algoritmo es "avaro" y solo mira la mejora inmediata de sus vecinos (visión en túnel), es extremadamente vulnerable a la "topografía" del espacio de estados. Tiene tres enemigos mortales:

Máximos locales

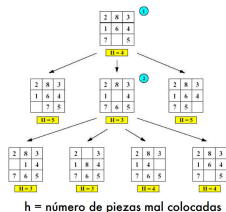
- Problema: El algoritmo llega a la cima de una colina. Todos los estados vecinos son "cuesta abajo", por lo que el algoritmo se detiene declarando victoria. Sin embargo, a unos kilómetros de distancia estaba el verdadero Máximo Global. Cuando aparecen cerca de la solución final, se llaman estribaciones.
- Solución: Backtracking (regresar a un estado anterior y explorar otra rama) o Random-Restart (si me atasco, me teletransporto aleatoriamente a otra parte del mundo y vuelvo a escalar; repito esto varias veces y me quedo con el pico más alto que haya encontrado).

Crestas (Ridges - el filo de la navaja)

- Problema: Una cresta es una montaña escarpada que sube en diagonal. Si los operadores del problema solo permiten moverse en cruz (Norte, Sur, Este, Oeste), intentar subir la cresta diagonal implica que cualquier paso en línea recta nos tire por el precipicio, haciendo creer al algoritmo que está en un máximo local, aunque visualmente la cresta siga subiendo.
- Solución: Permitir más de un operador antes de evaluar (por ejemplo, permitir evaluar el movimiento "Norte + Este" como un único paso diagonal).



Mesetas (Plateaus - el llano infinito)



- Problema: El algoritmo llega a una explanada donde el estado actual y todos sus vecinos tienen exactamente la misma puntuación heurística. El algoritmo no sabe a dónde ir porque nada mejora ni empeora. Se queda atascado vagando en círculos.
- Solución: Dar un gran salto en el espacio de búsqueda y tratar de alcanzar una región diferente del espacio de estados.

Limitaciones de la búsqueda clásica

Las estrategias vistas (anchura, profundidad, A*, etc) NO son adecuadas cuando:

- Las acciones tienen resultados inciertos
- El estado no se observa completamente
- El entorno cambia de forma independiente al agente

En estos casos:

- No se busca un único plan lineal
- Se requieren políticas, planes contingentes o estrategias de decisión

Más allá de la búsqueda determinista

Entornos no deterministas

- Búsqueda AND-OR; Planificación contingente

Entornos estocásticos

- Procesos de decisión de Markov (MDP); Aprendizaje por refuerzo

Entornos parcialmente observables

- Procesos de decisión de Markov parcialmente observables POMDPs; Creencias sobre estados

Entornos multiagente

- Búsqueda adversaria